[Serokell] GHC: Dependent types
Part 1: https://serokell.io/blog/ghc-dependent-types-in-haskell
Part 2: https://serokell.io/blog/ghc-dependent-types-in-haskell-2
Part 3: https://serokell.io/blog/ghc-dependent-types-in-haskell-3

Type level programming
Representing information in **types** instead of data.
In particular, **lifting** information that is usually represented at **term level** to **type level**.
By lifting to type level, program can be proved at **compile time**.

Term level	Type level
promoted data constructor	type i.e. type level **terms** e.g. type X = 'True 'True as type type X = 'Just 'False type X = '[1, 2] type X = 1 ': '[2]
promoted type constructor	kind i.e. type level **types** e.g. type X :: Bool Bool as kind type X = 'True
type family	type level functions e.g. type Not :: Bool -> Bool type family Not b where Not 'True = 'False Not 'False = 'True

Dependent types
A type where **term-level information** about a value is represented at **type-level**. i.e. **different values** may result in **different types**
e.g. a list type where the length is part of the type
Hence a list of length 0, 1, 2, 3 ... is each a different type

In a usual type, there is a **single type**, and various values are represented as **instances of the same type**.
In dependent types, there is a **family of types**, and each value may be presented by as an **instance of a different type** in the family.

```
-- traditional
data MyOption = MkMyOption Bool                  Every value is an instance of MyOption        i.e. different values of same type
x = MkMyOption True :: MyOption                  value= MkMyOption True   type= MyOption
y = MkMyOption False :: MyOption                 value= MkMyOption True   type= MyOption

-- dependent types
data MyOption (a:: Bool) = MkMyOption            Each value is a different type
x = MkMyOption :: MyOption 'True                 type = MyOption 'True
y = MkMyOption :: MyOption 'False                type = MyOption 'False          i.e. different types
```

Type level programming vs dependent types
Type level programming is about **representing** data as types.
Type level programming is a way of **implementing** dependent types, by

representing information about the **underlying value** as a type.
e.g.

True :: Bool	usual: True as a value, Bool as a type
'True :: Bool	type level programming: 'True **lifted** to a type, Bool lifted to a kind
MyData 'True	dependent type

Reflection vs Reification

Reflection	convert type -> term
Reification	convert term -> type

Type family as type level functions

Type families

Type family is a **mapping** from types to types. i.e. a **function** over types
Type family allows **computed types** i.e. a type computed by a function.

usage: computed output type
e.g. Not as a Bool -> Bool

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE GADTs #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TemplateHaskell #-}
{-# LANGUAGE TypeFamilies #-}

import Data.Singletons.TH

-- setup
genSingletons [''Bool]

-- BAD: infer without type family.
-- Doesn't work
notBad :: forall (a :: Bool) (b :: Bool). SingI b =>
Sing a -> Sing b
notBad STrue = SFalse
  -- ERROR. "Couldn't deduce b ~ False"

-- GOOD: with computed type
type Not :: Bool -> Bool
type family Not a where
  Not 'True = 'False
  Not 'False = 'True

sNot :: forall (a :: Bool). Sing a -> Sing (Not a)
sNot STrue = SFalse
  -- GOOD
sNot SFalse = STrue
  -- GOOD
```

aside: solution with opaque container

```
sNot2 :: forall (a :: Bool). Sing a -> SomeSing Bool
sNot2 STrue = SomeSing SFalse
```

```
sNot2 SFalse = SomeSing STrue
```

==========

Singleton

Data.Singleton: https://hackage.haskell.org/package/singletons
Introduction to singletons: https://blog.jle.im/entries/series/+introduction-to-singletons.html

Singleton
A singleton is a **term-level representation** of a **type parameter**. i.e. a unique value representing a type parameter.
In particular, it provides a to **pattern match a type parameter** at term level (see below).

Type parameter pattern matching
One of the uses of singletons is that it enables **type parameter pattern matching** types at **term-level.** i.e. pattern matching in a function by **type parameter**
It is possible to natively type pattern match at **type-level** (e.g. in a type family) not at term-level.

```
  -- Type parameter pattern match at type level
  type family Message (a :: Bool) :: Symbol where
Pattern match with type family. No need for singleton.
    Message 'True = "A"
    Message 'False = "B"
type X = Message 'True
type X = Message 'False


  -- Type parameter pattern match at term level
  --
  message :: forall (a :: Bool). Proxy a -> String                    BAD:
NO way to pattern match on parameter (a :: Bool) with proxy

  message (Proxy @'True) = "A"
ERROR

  message :: forall (a :: Bool). Sing a -> String
GOOD: Pattern match with singleton
  message STrue = "A"
  message SFalse = "B"
  x = message STrue
  x = message SFalse


  message :: forall (a :: Bool) -> String
GOOD: Pattern match with new dependent Haskell syntax (visible forall: https://github.com/ghc-proposals/ghc-proposals/blob/master/proposals/0281-visible-forall.rst#background)
  message (type 'True) = "A"
  message (type 'False) = "B"
  x = message (type 'True)
  x = message (type 'False)
```

==========
Decidability

[Tutorial] Introduction to singletons: https://blog.jle.im/entries/series/+introduction-to-singletons.html

Decidability
Witness (aka proof)
In type level programming, we may want to represent a certain **property of a type** e.g. a type-level integer is even
This is achieved by defining a **witness (aka proof)** of the property i.e. some representation such that

property B(a) is true <=> witness W(a) exists

The witness often needs to be computed **inductively** by a **decision function**.

Forms of properties

	Term as a witness	**Type** as witness	**Class instance** as witness
Description	Term exists <=> property is satisfied	e.g. type ~ 'True <=> property is satisfied	Instance exists <=> property is satisfied

Representation	`import Data.Type.Nat`
	
	`type IsEvenProof :: Nat -> Type`
	`data IsEvenProof n where ...`
		`import Data.Type.Nat`
		
		`type IsEven :: Nat -> Bool`
		`type family IsEven a where ...`
			`import Data.Type.Nat`
			
			`type IsEvenC :: Nat -> Constraint`
			`class IsEvenC n`

Witness	`data IsEvenProof n where`
	`  -- base witness: zero is even`
	`  EReflZ :: IsEvenProof 'Z`
	
	`  -- witness: if n is even, then n+2 is even`
	`  EReflSS :: IsEvenProof n`
	`  -> IsEvenProof ('S ('S n))`
		`type family IsEven a where`
		`  -- base witness: zero is even`
		`  IsEven 'Z = 'True`
		
		`-- witness: if n is even, then n+2 is even`
		`  IsEven ('S ('S a)) = IsEven a`
		`  IsEven _ = 'False`
			`-- base witness: zero is even`
			`instance IsEvenC 'Z where`
			
			`-- witness: if n is even, then n+2 is even`
			`instance IsEvenC n => IsEvenC ('S ('S n)) where`

Decision function	`-- Proof type IsEvenProof n`
	
	`-- decision function decideEven :: SNat n -> Dec (IsEvenProof n)`
		`-- Proof type (IsEven n :~: 'True)`
		
		`-- decision function decideEven2 :: SNat n -> Dec (IsEven n :~: 'True)`
			TODO

Using the property	`f :: IsEvenProof n -> SNat n -> ...`	`f :: (IsEven n ~ 'True) => SNat n -> ...`	`f :: (IsEvenC n) => SNat n -> ...`

Example : decide whether number is even/odd

```
{-# LANGUAGE DataKinds #-}
{-# LANGUAGE OverloadedStrings #-}
{-# LANGUAGE EmptyCase #-}
{-# LANGUAGE GADTs #-}
{-# LANGUAGE LambdaCase #-}
{-# LANGUAGE StandaloneKindSignatures #-}
{-# LANGUAGE TypeFamilyDependencies #-}
{-# LANGUAGE TypeOperators #-}
{-# OPTIONS_GHC -Werror=incomplete-patterns #-}
         -- IMPORTANT: ensure all cases are handled in
proofs

import Data.Kind
import Data.Type.Dec
import Data.Type.Nat
import Data.Type.Equality

snatPrev :: SNat ('S m) -> SNat m
snatPrev SS = snat
```

Term as a witness

```
  data IsEvenProof n where
    EReflZ :: IsEvenProof 'Z                                witness: 0 is
even
    EReflSS :: IsEvenProof n -> IsEvenProof ('S ('S n))     witness: if n
is even, then n+2
```

```
-- step: representation + witnesses
type IsEvenProof :: Nat -> Type

data IsEvenProof n where
```

```
  EReflZ :: IsEvenProof 'Z
  EReflSS :: IsEvenProof n -> IsEvenProof ('S ('S n))

-- step: decision function
decideEven :: SNat n -> Dec (IsEvenProof n)
                     -- decision function for
condition=IsEvenProof n with witness=IsEvenProof n
decideEven SZ = Yes EReflZ
decideEven s@SS = case snatPrev s of
                    -- induction over n with witness
IsEvenProof n
  SZ -> No $ \case {}
  t@SS -> case decideEven (snatPrev t) of
    Yes x -> Yes (EReflSS x)
    No rf -> No $ \case
      (EReflSS x) -> rf x


-- main: use
computeIfEven :: forall n. IsEvenProof n -> SNat n -> IO
()
computeIfEven _ n = putStrLn $ show (snatToNatural n) ++
" is even"

useSomeEven :: forall n. SNat n -> IO ()
useSomeEven n = case decideEven n of
                      -- decide
  Yes p -> computeIfEven p n
                          -- OK. n is even so computable
  No _ -> putStrLn $ show (snatToNatural n) ++ " is
ODD"                      -- NO. n ia not even so not
computable

-- e.g.
useSomeEven (snat @'Z)
useSomeEven (snat @Nat1)
useSomeEven (snat @Nat2)
```

Type family as a witness

```
 type isEven a where
   IsEven 'Z = 'True                          witness: 0 is even
   IsEven ('S ('S a)) = IsEven a              witness: if n is even, then n +
2 is even
   IsEven _ = 'False
```

```
-- step: representation + witness
type IsEven :: Nat -> Bool
type family IsEven a where
  IsEven 'Z = 'True
  IsEven ('S 'Z) = 'False
  IsEven ('S ('S a)) = IsEven a
```

```
-- step: decision function
--
-- Prove IsEven n ~ 'True with witness
--    IsEven n :~: 'True
--
decideEvenT :: SNat n -> Dec (IsEven n :~: 'True)

decideEvenT SZ = Yes Refl
decideEvenT n@SS = case snatPrev n of
  SZ -> No $ \case{}
  m@SS -> decideEvenT (snatPrev m)


-- usage
computeIfEvenT :: (IsEven n ~ 'True) => SNat n -> IO ()
computeIfEvenT n = putStrLn $ show (snatToNatural n) ++
" is even"

useSomeEvenT :: forall n. SNat n -> IO ()
useSomeEvenT n = case decideEvenT n of
  Yes Refl -> computeIfEvenT n
  No _ -> putStrLn $ show (snatToNatural n) ++ " is ODD"

-- e.g.
useSomeEven (snat @'Z)           -- "0 is even"
useSomeEven (snat @Nat1)         -- "1 is ODD"
useSomeEven (snat @Nat2)         -- "2 is even"
```

==========
OLD

Singleton
Types in type level programming are often **value less**. i.e do not have an associated value.
Singleton is a unified way of defining a corresponding **dummy value** for any types.

type <--singleton--> value

Singleton vs proxy
Singleton is a **typed proxy**
e.g.

```
import Data.Singletons
genSingletons [''Bool]

f :: SBool s -> ...             -- with singleton
```

```
g :: Proxy s -> ...             -- with proxy

f STrue
f (sing @'True)
g (Proxy @'True)
```

Decidability
In type level programming, a function can be called only if the given types **match** the function type signature.
A **decision function** determines whether the given types satisfy some **conditions** (aka **predicate**). e.g. type equality
A decision function is a **proof** of the condition.
A decision function often requires **induction** by inducting over a **witness** i.e. a value representing the property.
e.g.

```
-- A function to be called
f :: a -> Info a -> ...                          --
 Info = some type family
                                                 --
callable with f (x :: a) (y :: b)     only if b ~ Info a

-- Decision function
data a :~: b where                               --
witness b :~: Info a for condition b ~ Info a
  Refl :: forall k (a :: k). a :~: a

data Dec a where
 Yes a :: Dec a
 No Negated a :: Dec a
type Negated a = a -> Void

decide :: a -> b -> Dec (b :~: F a)              --
decide = decision function for condition b ~ Info a with
witness b :~: Info a
decide x y = ... <Proof>


-- e.g. usage
main :: a -> b -> ...
main x y = case decide a b of
  Yes Refl -> f x y                            -- OK.
Can call f since b ~ F a
  No _ -> ...                                    -- NO.
Can't  call f due to type mismatch
```

Proof
In most cases, the compiler can NOT **infer** the type property, but the decision function needs to **prove** the type property to the compiler.
Proof is often by **induction over the types** using a **witness** i.e. a **value** to carry the property through the induction.
e.g. inference failure

```
import Data.Type.Dec
import Data.Type.Nat

snatPrev :: SNat ('S m) -> SNat m
snatPrev SS = snat

decide :: SNat n -> Dec (SomeProperty n)
decide SS = Refl                                    --
ERROR "Couldn't match expected type Dec (SomeProperty
n)"
decide s@SS = case decide (snatPrev s) of           --
OK. Proof by induction on n
  Yes ... -> ...
  No ... -> ...
```

Predicate (type predicate)
A **(type) predicate** represents a **property** of a **type**.
If the predicate is **computable** i.e. has a value, then the property is satisfied.
Given a type, compiler usually can NOT infer computability but need to be **proved** by the program.
The proof is provided by a **decision function**.
The proof typically involves **induction** over a **witness** i.e. a value representing the property
e.g.

```
f :: a -> F a -> ...

data a :~: b where
  Refl :: n :~: n

decide :: a -> b -> Dec (b ~ F a)

run :: a -> b -> ...
run a b = case decide a b of
  Yes Refl -> f a b
  No _ ->
```

e.g. components of a predicate

```
import Data.Type.Nat
import Data.Type.Dec

-- 1) predicate IsZero n
type IsZero n = n ~ 'Z
computeIfZero :: IsZero n => SNat n -> ...
         -- e.g. a function callable only with zero

-- 2) witness for predicate IsZero n
data n :~: m where
  Refl :: n :~: n
```

```
-- 3) decision function for IsZero n with witness
n :~: 'Z
decide :: SNat n -> Dec (n :~: 'Z)
decide SZ = Yes Refl
decide SS = No $ \case{}

-- e.g. usage
f :: SNat n -> ...
f n = case decide n of
 Yes Refl -> computeIfZero ...
 No _
```